# Day 4 — Advanced Java Programming Concepts

## Topics

- Arrays:-

An **array in Java** is a collection of elements of the same data type stored under a single variable name. Array elements are accessed using an index, starting from `0`.

An **array** in Java is a fixed-size, homogenous data structure that stores a collection of elements of the same data type in contiguous memory locations.

- Strings:-

An Java, a `String` is an object that represents a sequence of characters (like text). Unlike primitive data types (such as `int` or `char`), `String` is a reference type backed by the `java.lang.String` class.

---

- Collections Framework:-

The **Java Collections Framework (JCF)** is a unified architecture for storing and manipulating groups of objects. It provides pre-built data structures (like lists, sets, and maps) and algorithms to high performance and reduce programming effort.

## Core Hierarchy of the Collections Framework

The framework is divided into two distinct interface trees:

1. **The `Collection` Interface**: The root interface for standard data structures (Lists, Sets, Queues).
2. **The `Map` Interface**: A separate structure for key-value pairs (does not extend `Collection`).

- ArrayList and HashMap :-

An **`ArrayList` in Java** is a resizable, dynamic array implementation of the `List` interface found in the `java.util` package. Unlike standard arrays which have a fixed length, an `ArrayList` automatically grows in size as elements are added and shrinks when elements are removed.

An `ArrayList` is like a resizable array.

It is part of the `java.util` package and implements the `List` interface.

The difference between a built-in array and an `ArrayList` in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an `ArrayList` whenever you want.

## *HashMap:-*

A `HashMap` stores items in **key/value pairs**, where each key maps to a specific value.

It is part of the `java.util` package and implements the `Map` interface.

Instead of accessing elements by an index (like with ArrayList), you use a **key** to retrieve its associated **value**.

A `HashMap` can store many different combinations, such as:

- `String` keys and `Integer` values
- `String` keys and `String` values

## Exception handling:-

Exception handling in Java is a mechanism used to handle exceptional conditions that occur during program execution. It helps prevent the program from terminating unexpectedly and allows developers to provide appropriate recovery or error-handling logic.

- Separates error-handling code from normal program logic.
- Helps maintain the normal flow of program execution.
- Supports handling of both checked and unchecked exceptions.

## Java Exception Hierarchy

In Java, all exceptions and errors are subclasses of the Throwable class. It has two main branches

1. Exception.
2. Error

# 1. Types of exceptions in Java

- **Checked exceptions:** Checked by the compiler, such as `IO Exception`.
- **Unchecked exceptions:** Occur at runtime, such as `Arithmetic Exception`, `NullPointer Exception`, and `ArrayIndexOutOfBoundsException`.

- **Errors:** Serious problems generally not intended to be handled by ordinary application code, such as `OutOfMemoryError`.

## • Introduction to Java Streams

In Java, a **Stream** is a sequence of elements supporting sequential and parallel aggregate operations. Introduced in Java 8, streams allow you to process data collections (like lists or sets) declaratively, similar to how you write SQL queries, rather than using complex loops.

One of the major new features in Java 8 is the introduction of the stream functionality – *java.util.stream* – which contains classes for processing sequences of elements.

The central API class is the *Stream<T>*. The following section will demonstrate how streams can be created using the existing data-provider sources.

## Tasks

- Store and process student or employee records.
- Search for a record by ID or name.
- Sort records by name or marks.
- Handle invalid input using exception handling.
- Practice working with ArrayList and HashMap.

Java program:-

```
import java.util;

Class Student {
    int id;
    String name;
    double marks;

    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    void display() {
        System.out.println(
            "ID: " + id +
            ", Name: " + name +
", Marks: " + marks
        );
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Student> students = new ArrayList<>();
        HashMap<Integer, Student> studentMap = new HashMap<>();

        try {
            System.out.print("Enter number of students: ");
            int n = Integer.parseInt(sc.nextLine());

            if (n <= 0) {
throw new IllegalArgumentException(
                    "Number of students must be positive."
                );
            }

            // Input student records
            for (int i = 0; i < n; i++) {
                System.out.println("\nStudent " + (i + 1));

                System.out.print("Enter ID: ");
                int id = Integer.parseInt(sc.nextLine());

                if (studentMap.containsKey(id)) {
                    throw new IllegalArgumentException(
                        "Duplicate student ID."
                    );
                }

                System.out.print("Enter name: ");
                String name = sc.nextLine().trim();

if (name.isEmpty()) {
                    throw new IllegalArgumentException(
                        "Name cannot be empty."
                    );
                }

                System.out.print("Enter marks (0-100): ");
                double marks = Double.parseDouble(sc.nextLine());

                if (marks < 0 || marks > 100) {
                    throw new IllegalArgumentException(
                        "Marks must be between 0 and 100."
                    );
                }

                Student s = new Student(id, name, marks);
                Students.add;
studentMap.put (id, s);
            }

            // Display all records
            System.out.println("\n--- All Student Records ---");
```

```
            for (Student s : students) {
                s.display();
            }

            // Search by ID
            System.out.print ("\enter ID to search: ");
            Int search ID = Integer.parseInt(sc.nextLine());

            Student found = studentMap.get (searched);

            If (found!= null) {
                System.out.println ("Record found :");
                Found. Display ();
} else {
                System.out.println ("Student not found.");
            }

            // Search by name
            System.out.print ("\enter name to search: ");
            String search Name = sc.nextLine ().trim ();
            Boolean nameFound = false;

            For (Student s: students) {
                If (s.name.equalsIgnoreCase (search Name)) {
                    s.display();
                    nameFound = true;
                }
            }

            If (! nameFound) {
                System.out.println("Student not found.");
            }

            // Sort by name
            students.sort(
                Comparator.comparing(
                    S -> s.name.toLowerCase ()
)
            );

            System.out.println("\n--- Sorted by Name ---");
            For (Student s: students) {
                s.display();
            }

            // Sort by marks (descending)
            students.sort(
                Comparator.comparingDouble(
                    (Student s) -> s.marks
                ).reversed()
            );

            System.out.println("\n--- Sorted by Marks ---");
            for (Student s : students) {
                s.display();
}
```

```
        } catch (NumberFormatException e) {
            System.out.println(
                "Invalid input! Please enter a valid number."
            );
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

Output:-

```
Enter number of students: 3

Student 1
Enter ID: 101
Enter name: Rahul
Enter marks (0-100): 85

Student 2
Enter ID: 102
Enter name: Priya
Enter marks (0-100): 92

Student 3
Enter ID: 103
Enter name: Amit
Enter marks (0-100): 78

--- All Student Records ---
ID: 101, Name: Rahul, Marks: 85.0
ID: 102, Name: Priya, Marks: 92.0
ID: 103, Name: Amit, Marks: 78.0
```